Claude Code Analytics Platform

Making Sense of Developer Telemetry Data

100 Engineers | 5,000 Sessions | 454K Events | 60 Days

Malek Aldroubi

Provectus Gen AI Internship, Technical Assessment

The Problem

What we're starting with and what we need to get to

The Starting Point

- A 521 MB JSONL file with 454K telemetry events, not structured for analysis.
- Events are nested inside CloudWatch-style log batches, requiring multi-level parsing.
- 5 different event types mixed together: API calls, tool usage, prompts, errors.
- Employee metadata lives in a separate CSV with no direct link to the events.
- Stakeholders want answers: Who spends the most? What's failing? When do people code?

What the Raw Data Looks Like

telemetry_logs.jsonl (521 MB)

```
{"messageType":"DATA_MESSAGE", "logEvents":[
  {"message": "{\"body\":\"claude_code.api_request\",
    \"attributes\": {\"model\":\"claude-opus-4-6\",
      \"cost_usd\":\"0.071\", \"duration_ms\":\"10230\",
      \"input_tokens\":\"263\", ...}}"},
  {"message": "{\"body\":\"claude_code.tool_result\", ...}"}
]}
```

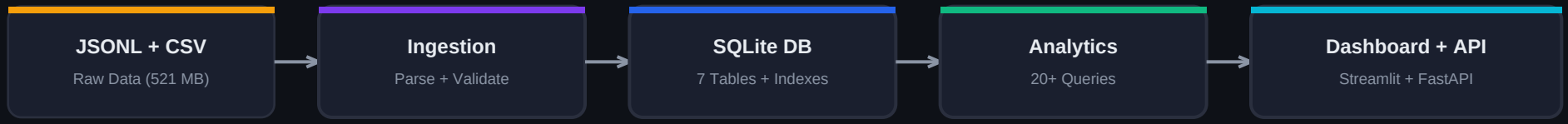
What We Need to Build

- A clean, queryable database
- Cost tracking by model, team, and user
- Usage pattern analysis (when, how, who)
- Tool performance monitoring
- Anomaly detection for cost spikes
- An interactive dashboard anyone can use

How I Built It

Breaking the problem into manageable steps

Pipeline Overview



What I Did at Each Step

1 Explored the Data

Read through the JSONL to understand the nested structure. Identified 5 event types, each with different fields. Mapped how events connect to employee metadata.

2 Designed the Database

Chose normalized tables (one per event type) instead of a flat table, because the queries I needed are different for each type. Added indexes on the columns I filter most.

3 Built the Ingestion Pipeline

Parses nested JSONL, validates each event, and batch-inserts into SQLite. Optimized with WAL mode and 5000-row flushes. Processes 454K events in about 20 seconds.

4 Wrote the Analytics Layer

20+ query functions organized by topic: cost breakdowns, usage trends, model comparison, tool success rates, error tracking, and anomaly detection.

5 Built the Dashboard

8-page Streamlit app with Plotly charts. Designed each page around a specific question: where does the money go, when do people code, what's failing, etc.

6 Added Bonus Features

REST API with 15 endpoints for programmatic access. Anomaly detection using rolling statistics to flag unusual cost spikes at daily and per-user levels.

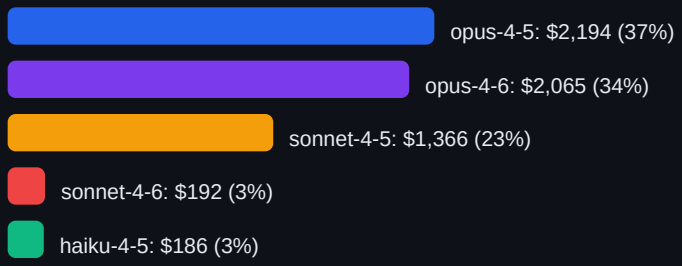
Built with: Python 3 | SQLite | Streamlit | Plotly | FastAPI | pandas

What the Data Tells Us

Key findings from the analysis

100	5,000	$6,001	1.2%	$1.20
Active Users	Total Sessions	Total Cost	Error Rate	Avg $/Session

Where the Money Goes



Interesting Patterns

- Most activity happens at 17:00 UTC
- About 70% of work is during business hours
- Read and Bash are the most-used tools
- Bash has the lowest success rate (93.0%)
- File operations (Read, Edit) almost never fail
- All 100 engineers are actively using the platform

BIGGEST TAKEAWAY

Opus models are 71% of the total cost but handle only 42% of requests. If simple tasks were routed to Haiku ($0.004/req instead of $0.09/req), the company could cut spending by 50% or more.

Bonus Features & Next Steps

What I added and where this could go

Bonus: Anomaly Detection

Uses a 7-day rolling average with 2 standard deviations as the threshold

- Found 1 unusual day(s) out of 60 in the dataset.
- Flagged 50 cases where individual users spent way above their norm.
- Most spikes come from single long coding sessions, not systemic problems.
- Overall the platform looks healthy, no sustained cost drift detected.

Bonus: REST API

- 15 endpoints covering every analytics domain (cost, tools, models, users, anomalies).
- Same data as the dashboard, accessible programmatically.
- Could be used for automated weekly reports or integration with other tools.

If I Had More Time, I'd Add

- **Smart Model Routing**
  Automatically suggest Haiku for simple tasks and Opus for complex ones. Could save 50%+ on cost.
- **Cost Alerts**
  Set a daily budget per user (e.g. $15) and flag when someone exceeds it, before the bill adds up.
- **Bash Error Analysis**
  Dig into why Bash fails 7% of the time. Are there common error patterns we can fix?
- **Weekly Email Reports**
  Use the API to auto-generate a summary for managers every Monday.
- **Live Streaming**
  Connect to a real telemetry stream instead of batch processing historical data.

454K events in 20s | 7 database tables | 20+ analytics queries | 8 dashboard pages | 15 API endpoints | Anomaly detection